

3. Once you are sure that the downloaded tarball is not corrupt, you can extract its content and start the compilation (please consider that the extracted archive will take around 200 MB of disk space, and the compilation will take up some extra space):

```
$ tar xjvf postgresql-16.0.tar.bz2
$ cd postgresql-16.0
$ ./configure --prefix=/usr/local
$ make && sudo make install
```

If you want or need the `systemd(1)` service file, add the `--with-systemd` option to the `configure` line.

4. Once the database has been installed, you need to create a user to run the database with, usually named `postgres`, and initialize the database directory:

```
$ sudo useradd postgres
$ sudo mkdir -p /postgres/16/data
$ sudo chown -R postgres:postgres /postgres/16
$ /usr/local/bin/initdb -D /postgres/16/data
```

Installing PostgreSQL via pgenv

`pgenv` is a nice and small tool that allows you to download and manage several instances of different versions of PostgreSQL on the same machine. The idea behind `pgenv` is to let you explore different PostgreSQL versions—for instance, to test your application against different major versions. `pgenv` does not aim to be an enterprise-class tool to manage in-production instances; rather, it is a tool to let developers and DBAs experiment with different versions of PostgreSQL and keep them under control easily.

Of course, being an external tool, `pgenv` must be installed before it can be used. The installation, however, is very simple, since the application is made by a single Bash script.

The fastest way to get `pgenv` installed is to clone the GitHub repository and set the `PATH` environment variable to point to the executable directory, as follows:

```
$ git clone https://github.com/theory/pgenv
$ export PATH=$PATH:./pgenv/bin
```

Now, the `pgenv` command is at your fingertips, and you can run the command to get a help prompt and see the available commands.

The idea behind pgenv is pretty simple: it is a tool to automate the “boring” stuff—that is, downloading, compiling, installing, and starting/stopping a cluster. In order to let pgenv manage a specific instance, you have to “use” it. When you use an instance, pgenv detects whether the instance has been initialized or not, and in the latter case, it does the initialization for you.

In order to install versions 16.0 and 15.1 of PostgreSQL, you simply have to run the following commands:

```
$ pgenv build 16.0  
  
$ pgenv build 15.1
```

The preceding commands will download and compile the two versions of PostgreSQL, and the time required for the operations to complete depends on the power and speed of the machine you are running on. After that, you can decide which instance to start with the use command:

```
$ pgenv use 16.0
```

pgenv is smart enough to see whether the instance you are starting has already been initialized, or it will initialize it (only the first time) for you.

If you need to stop and change the PostgreSQL version to use, you can issue a stop command followed by a use command with the targeted version. For instance, to stop running the 16.0 instance and start a 15.1 instance, you can use the following:

```
$ pgenv stop  
  
$ pgenv use 15.1
```

The pgenv tool provides a lot of other commands to get information about which PostgreSQL versions are installed, what is executing (if any), and so on.

If you are searching for a quick way to test and run different PostgreSQL versions on the same machine, pgenv is a good tool.

Summary

This chapter has introduced you to PostgreSQL, the project, and its main features. You have learned about PostgreSQL terminology, as well as how to install a cluster on Unix-like operating systems, including in containers, as well as installing the cluster from various sources.

Having installed PostgreSQL and having learned its terminology allows you to proceed to the next chapters, where you will learn how to use, connect, and store data in a database.

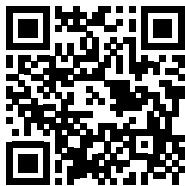
References

- PostgreSQL release notes: <https://www.postgresql.org/docs/16/release-16.html>
- Upgrading documentation: <https://www.postgresql.org/docs/current/upgrading.html>
- PostgreSQL version policy: <https://www.postgresql.org/support/versioning/>
- PostgreSQL initdb official documentation: <https://www.postgresql.org/docs/current/app-initdb.html>
- PostgreSQL pg_ctl official documentation: <https://www.postgresql.org/docs/current/app-pg-ctl.html>
- pgenv GitHub repository and documentation: <https://github.com/theory/pgenv>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



2

Getting to Know Your Cluster

To be a proficient user and administrator of a PostgreSQL cluster, you first must know and understand how PostgreSQL works. A database system is a very complex beast, and PostgreSQL, being an enterprise-level **Database Management System (DBMS)**, is in no way a simple software system. However, thanks to very good design and implementation, once you understand the basic concepts and terminology of PostgreSQL, things will quickly become comprehensive and clear.

This chapter will continue from the foundation of the previous chapter and introduce you to some other PostgreSQL terminology and concepts, as well as teaching you how to interact with the cluster. You will also be introduced to the `psql` client, which ships with PostgreSQL and is the recommended way to connect to your database. You are free to use any SQL client that can connect to PostgreSQL, and all the code and examples shown in this chapter will run out of the box in any other client as well, but we recommend that you take some time to learn `psql`. Shipped with PostgreSQL, `psql` is guaranteed to work in any situation and is the default way to connect to a cluster. `psql` is a text-only client; if you are more comfortable using a graphical client, you can have a look at `pgAdmin4`, one of the most famous PostgreSQL graphical clients.

This chapter covers the following main topics:

- Managing your cluster
- Connecting to the cluster
- Exploring the disk layout of PGDATA
- Exploring configuration files and parameters

Technical requirements

The knowledge required in this chapter is as follows:

- How to install binary packages on your Unix machine
- PostgreSQL basic terminology (from the previous chapter)
- Basic Unix command-line usage
- Basic SQL statements covered in this chapter, like `SELECT`

The chapter examples can be run on the standalone Docker image, which you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>. For installation and usage of the Docker images available for this book, please refer to the instructions in *Chapter 1, Introduction to PostgreSQL*.

Managing your cluster

A PostgreSQL cluster is a collection of several databases that all run under the very same PostgreSQL service or instance.

Managing a cluster means being able to start, stop, take control, and get information about the status of a PostgreSQL instance.

From an operating system point of view, PostgreSQL is a service that can be started, stopped, and, of course, monitored. As you saw in the previous chapter, usually when you install PostgreSQL, you also get a set of operating system-specific tools and scripts to integrate PostgreSQL with your operating system service management. Usually, you will find system service files or other operating system-specific tools, like `pg_ctl cluster`, which is shipped with Debian GNU/Linux and its derivatives.

PostgreSQL ships with a specific tool called `pg_ctl`, which helps in managing the cluster and the related running processes. This section introduces you to the basic usage of `pg_ctl` and to the processes that you can encounter in a running cluster. It does not matter which service management system your operating system is using, `pg_ctl` will always be available to the PostgreSQL administrator in order to take control of a database instance.

`pg_ctl`

The `pg_ctl` command-line utility allows you to perform different actions on a cluster, mainly initialize, start, restart, stop, and so on. `pg_ctl` accepts the command to execute as the first argument, followed by other specific arguments—the main commands are as follows:

- `start`, `stop`, and `restart` execute the corresponding actions on the cluster.

- `status` reports the current status (running or not) of the cluster.
- `initdb` (or `init` for short) executes the initialization of the cluster, possibly removing any previously existing data.
- `reload` causes the PostgreSQL server to reload the configuration, which is useful when you want to apply configuration changes.
- `promote` is used when the cluster is running as a replica server (namely a standby node) and, from now on, must be detached from the original primary becoming independent (replication will be explained in later chapters).

Generally speaking, `pg_ctl` interacts mainly with the `postmaster` (the first process launched within a cluster), which in turn “redirects” commands to other existing processes. For instance, when `pg_ctl` starts a server instance, it makes the `postmaster` process run, which in turn completes all the startup activities, including launching other utility processes (as briefly explained in the previous chapter). On the other hand, when `pg_ctl` stops a cluster, it issues a `halt` command to the `postmaster`, which in turn requires other active processes to exit, waiting for them to finish.



The `postmaster` process is just the very first PostgreSQL-related process launched within the instance; on some systems, there is a process named “`postmaster`,” while on other operating systems, there are only processes named “`postgres`.” The first process ever launched, despite its name, is referred to as the `postmaster`. The name `postmaster` is just that, a name used to identify a process among the others (in particular, the *first* process launched within the cluster).

`pg_ctl` needs to know where the PGDATA is located, and this can be specified by either setting an environment variable named PGDATA or by specifying it on the command line by means of the `-D` flag.

Interacting with a cluster status (for example, to stop it) is an action that not every user must be able to perform; usually, only an operating system administrator must be able to interact with services including PostgreSQL.

PostgreSQL, in order to mitigate the side effects of privilege escalation, does not allow a cluster to be run by privileged users, such as `root`. Therefore, PostgreSQL is run by a “normal” user, usually named `postgres` on all operating systems. This unprivileged user owns the PGDATA directory and runs the `postmaster` process, and, therefore, also all the processes launched by the `postmaster` itself. `pg_ctl` must be run by the same unprivileged operating system user that is going to run the cluster.



If you are using the Docker image, PostgreSQL is already running as the main service. This means that issuing a `stop` or a `restart` command will force you to exit from the container due to its shutdown.

Moreover, in the Docker container, the PostgreSQL service will be already running without any need for manual intervention.

The `status` command just queries the cluster to get information, so it is pretty safe as a starting point to understand what is happening:

```
$ pg_ctl status
pg_ctl: server is running (PID: 1)
/usr/lib/postgresql/16/bin/postgres
```

The command reports back that the server is running, with a **Process Identifier (PID)** equal to one (this number will be different on your machine). Moreover, the command reports the executable file used to launch the server, in the above example, `/usr/lib/postgresql/16/bin/postgres`.

If the server is not running for any reason, the `pg_ctl` command will report an appropriate message to indicate that is unable to find an instance of PostgreSQL started:

```
$ pg_ctl status
pg_ctl: no server running
```

In order to report the status of the cluster, `pg_ctl` needs to know where the database is storing its own data—that is, where the PGDATA is on disk. There are two ways to make `pg_ctl` aware of where the PGDATA is:

- Setting an environment variable named `PGDATA`, containing the path of the data directory
- Using the `-D` command-line flag to specify the path to the data directory



Almost every PostgreSQL cluster-related command searches for the value of `PGDATA` as an environmental variable or as a `-D` command-line option.

In the previous examples, no `PGDATA` has been specified, and this is because it has been assumed the value of the `PGDATA` was specified by an environment variable.

It is quite easy to verify this—for example, in the Docker container:

```
$ echo $PGDATA
/postgres/16/data
$ pg_ctl status
pg_ctl: server is running (PID: 1)
/usr/lib/postgresql/16/bin/postgres
```

In the case that your setup does not include an PGDATA environment variable, you can always set it manually before launching `pg_ctl` or any other cluster-related command:

```
$ export PGDATA=/postgres/16/data
$ pg_ctl status
pg_ctl: server is running (PID: 1)
```

The command-line argument, specified with `-D`, always has precedence against any PGDATA environment variable, so if you don't set or misconfigure the PGDATA variable but, instead, pass the right value on the command line, everything works fine:

```
$ export PGDATA=/postgres/data # wrong PGDATA!
$ pg_ctl status -D /postgres/16/data
pg_ctl: server is running (PID: 1)
/usr/lib/postgresql/16/bin/postgres "-D" "/postgres/16/data"
```

The same concepts of PGDATA and the `-D` optional argument are true for pretty much any “low-level” commands that act against a cluster and make clear that, with the same set of executables, you can run multiple instances of PostgreSQL on the same machine, as long as you keep the PGDATA directory of each one separate.



Do not use the same PGDATA directory for multiple versions of PostgreSQL. While it could be tempting, on your own test machine, to have a single PGDATA directory that can be used in turn by a PostgreSQL 16 and a PostgreSQL 15 instance, this will not work as expected and you risk losing all your data. Luckily, PostgreSQL is smart enough to see that PGDATA has been created and used by a different version and refuses to operate, but please be careful not to share the same PGDATA directory with different instances.

`pg_ctl` can be used to start and stop a cluster by means of appropriate commands. For example, you can start an instance with the `start` command (assuming a `PGDATA` environment variable has been set):

```
$ pg_ctl start
waiting for server to start....
[27765] LOG:  starting PostgreSQL 16.0 on x
86_64-pc-linux-gnu, compiled by gcc (GCC) 12.1.0, 64-bit
[27765] LOG:  listening on IPv6 address ":::1", port 5432
[27765] LOG:  listening on IPv4 address "127.0.0.1", port 5432 [27765]
LOG:  listening on Unix socket "/tmp/.s.PGSQL.5432"
[27768] LOG:  database system was shut down at 2023-07-19 07:20:24 EST
[27765] LOG:  database system is ready to accept connections
done
server started
```



The `start`, `stop`, and `restart` commands do not work on the Docker images from this book's repository because such containers are running PostgreSQL as the main process; therefore, stopping (or restarting) will cause the container to exit. Similarly, there is no need to start the service because it is automatically started once the container starts.

The `pg_ctl` command launches the `postmaster` process, which prints out a few log lines before redirecting the logs to the appropriate log file. The `server started` message at the end confirms that the server has started. During the startup, the PID of the `postmaster` is reported within square brackets; in the above example, the `postmaster` is the operating system process number 27765.

Now, if you run `pg_ctl` again to check the server, you will see that it has been started:

```
$ pg_ctl status
pg_ctl: server is running (PID: 27765)
/usr/pgsql-16/bin/postgres
```

As you can see, the server is now running and `pg_ctl` shows the PID of the running `postmaster` (27765), as well as the executable command line (in this case, `/usr/pgsql-16/bin/postgres`).



Remember: The postmaster process is the first process ever started in the cluster. Both the backend processes and the postmaster are run starting from the `postgres` executable, and the postmaster is just **the root of all PostgreSQL processes**, with the main aim of keeping all the other processes under control.

Now that the cluster is running, let's stop it. As you can imagine, `stop` is the command used to instruct `pg_ctl` about which action to perform:

```
$ pg_ctl stop
waiting for server to shut down....
[27765] LOG:  received fast shutdown request
[27765] LOG:  aborting any active transactions
[27765] LOG:  background worker "logical replication launcher" (PID 27771)
exited with exit code 1
[27766] LOG:  shutting down
[27766] LOG:  checkpoint starting: shutdown immediate
[27766] LOG:  checkpoint complete: wrote 0 buffers (0.0%); 0 WAL file(s)
added, 0 removed, 0 recycled; write=0.001 s, sync=0.001 s, total=0.035
s; sync files=0, longest=0.000 s, average=0.000 s; distance=0 kB,
estimate=237 kB; lsn=0/1529DC8, redo lsn=0/1529DC8
[27765] LOG:  database system is shut down
done
server stopped
```

During a shutdown, the system prints a few messages to inform the administrator about what is happening, and as soon as the server stops, the message `server stopped` confirms that the cluster is no longer running.

Shutting down a cluster can be much more problematic than starting it, and for that reason, it is possible to pass extra arguments to the `stop` command in order to let `pg_ctl` act accordingly. There are three ways of stopping a cluster:

- The smart mode means that the PostgreSQL cluster will gently wait for all the connected clients to disconnect and only then will it shut the cluster down.
- The fast mode will immediately disconnect every client and will shut down the server without having to wait.